COMPUTER'S LANGUAGE DECISION MECHANISM

Computers generally use three fundamental academic approaches to determine the language of a text.

1.1. N-Gram Character Statistics : In this method, the text is divided into character sequences rather than words. Each language has its unique character combinations. For example:

- Sequences like **"th", "wh", "tion"** are frequently seen in English.
- Sequences like **"ğü", "şç", "ıo"** are specific to Turkish.

The computer splits the text into groups of 2 (bigrams) or 3 (trigrams) characters and compares the frequency of these patterns with pre-established language profiles.

1.2. Stop-Words Analysis: Every language contains functional words that are used very frequently. Words like **"the", "is", "and", "in"** in English, and **"ve", "bir", "bu", "da"** in Turkish, play a significant role in sentence structure. The density of these words within a text provides a strong signal for language prediction. However, modern language detection systems typically evaluate these words indirectly through character-level statistical models rather than counting them directly.

1.3. Probabilistic Language Models

Statistical models, which have been previously trained on millions of sentences, determine the language of a given text by comparing it against learned language vectors. Through this comparison, the system generates a probability value commonly referred to as a **"confidence score."** For instance, the model might conclude that a specific text is 98% likely to be English. This vector-based approach ensures a high degree of accuracy, particularly when processing large-scale datasets.

2.LANGUAGE DETECTION

Language detection is the process of automatically determining the language of a given text. This process is carried out using statistical profiling methods rather than dictionary-based approaches. Among the widely used libraries for language detection in the Python ecosystem are langdetect, langid.py, and fastText.These libraries produce a probabilistic language label by analyzing the character distribution and statistical.

2.1.The langdetect Library: langdetect is a popular Python library developed based on Google's language detection project, designed to identify the language of a given text. Supporting over 50 languages, this tool is well-known for providing fast and reliable results, particularly in complex texts. Its working mechanism is as follows: it analyzes the text by breaking it down into small character groups called N-Grams. By comparing these segments with language profiles in its own database, the library conducts a probabilistic analysis and returns the language label (e.g., TR, EN, FR) with the highest confidence score as the final result.

2.2. The langid.py Library

langid.py is a standalone language identification tool that aims to provide high-speed performance and ease of use without requiring any external dependencies. Unlike many other libraries, it comes with a pre-trained model integrated into a single Python script, making it highly portable and efficient for large-scale data processing. Its algorithm is specifically optimized to maintain high accuracy across a wide range of languages while minimizing computational overhead. Therefore, *langid.py* is often the preferred choice in environments where processing speed is a critical factor or where a lightweight, self-contained solution is needed.

2.3. The fastText Library

fastText is a high-performance classification model developed by Meta AI, based on character-level n-gram representations. Unlike traditional word-based models, it represents each word as a bag of character n-grams, allowing it to understand the structure of words and perform exceptionally well even on rare or out-of-vocabulary words.The key features of the model include:

- **Broad language support:** A pre-trained model (lid.176) capable of recognizing 176 different languages.
- **N-gram based representation:** Analyzes character sequences to capture the morphological nuances of languages.
- **Efficiency:** Offers high-speed processing with low computational cost, making it ideal for real-time applications.
- **Probabilistic scoring:** Generates a confidence score for each prediction, outputting the language with the highest probability.

3. DATA PRE-PROCESSING AND PREPARATION

3.1. Sampling

In this study, a sampling method was employed to determine the language distribution of the large dataset consisting of 65,000 rows and to test the model's performance. The "sample size for a finite population" formula was used to calculate the required sample size, as the total population size is known:

In this formula; N (65,000) represents the population size, Z (1.96 for a 95% confidence level) represents the confidence coefficient, P (0.5) represents the maximum variance probability since the language homogeneity of the population is unknown, and d(0.02) represents the acceptable margin of error. Based on these calculations, a sample size of approximately 2,318 units was determined to represent the population with a 95% confidence level and a 2% margin of error. To ensure an equal probability of selection for each review and to prevent bias, the "simple random sampling" technique was applied during the selection phase. This approach has formed the basis for verifying the accuracy of the language detection model and identifying the proportion of non-English reviews (outliers) within a statistical confidence interval.

3.2. Text Cleaning

Before applying language detection, performing a thorough text cleaning process is essential to ensure consistency across the dataset. During this phase, case differences are eliminated through lowercasing, and noise-inducing elements such as emojis and HTML tags (e.g.,
) are stripped from the text. Additionally, redundant whitespaces and tab characters are corrected, while excessive punctuation is simplified to create a more uniform structure. These operations are typically executed using Regular Expressions (Regex) in Python. Ultimately, this cleaning process enhances the model's accuracy rate by allowing for a more precise analysis of the character distribution within the text.

3.3. Outlier Detection

In this study, comments written in languages other than English are classified as outliers within the dataset. Following the language detection process, these non-English comments are identified and systematically removed before the final analysis. This procedure significantly reduces the noise that multiple languages might introduce during model training, ensuring a more focused and accurate dataset for the subsequent stages of the project.